

Rapport de Projet DevOps : Déploiement et Orchestration d'un LLM Open-Source

Auteur: Médoune DIOP, DIC2-GIT

Module: Cloud & DevOps

Professeur: M. Ibrahima MBENGUE

1. Introduction et Contexte

L'essor des modèles de langage ouverts (*Open-Source Large Language Models - LLMs*) tels que LLaMA, Mistral ou DeepSeek offre aux organisations une alternative souveraine et économique aux API propriétaires cloud (OpenAI, Google Gemini).

Cependant, le déploiement d'applications d'IA générative soulève des contraintes techniques et opérationnelles significatives :

- **Ressources matérielles intensives** : Forte sollicitation CPU/RAM lors de l'inférence.
- **Poids des modèles** : Nécessité de gérer des artefacts volumineux (plusieurs Go par modèle).
- **Temps de démarrage** : Délai de chargement des poids en mémoire vive avant disponibilité de l'API.
- **Résilience** : Exigence de haute disponibilité et d'auto-guérison en production.

Objectif du projet

1. Déployer localement une architecture multi-conteneurs pour LLM via **Docker Compose**.
2. Assurer sa migration complète vers **Kubernetes (Minikube)** en appliquant les bonnes pratiques DevOps (isolation, quotas, sondes de santé, persistance et exposition réseau).

2. Architecture Globale de l'Application

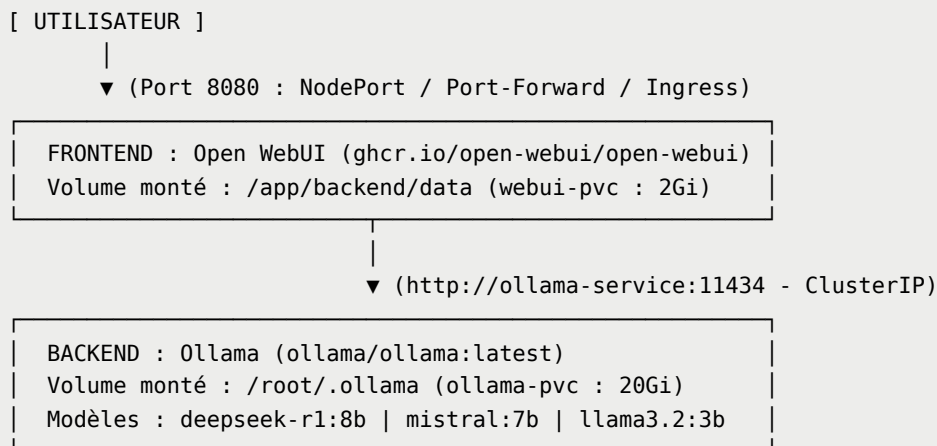
L'application est architecturée autour de deux composants indépendants et découplés :

1. Ollama (Backend d'Inférence LLM)

Moteur d'inférence haute performance basé sur `llama.cpp`. Il expose une API REST sur le port interne `11434`, gère le téléchargement, la quantification et l'exécution locale des modèles de langage.

2. Open WebUI (Frontend Utilisateur)

Interface web moderne reproduisant l'expérience utilisateur de ChatGPT. Elle gère l'authentification, l'historique des conversations, le streaming des réponses en temps réel et dialogue avec Ollama via l'API REST.



3. Partie 1 – Déploiement Local avec Docker Compose

3.1. Structure de la configuration

Le fichier `docker-compose.yaml` orchestre les conteneurs sur la machine hôte :

- **Service `ollama`** : Expose le port hôte `11434`, monte un volume nommé `ollama` vers `/root/.ollama` et configure un healthcheck (`ollama list`).

- **Service `open-webui`** : Expose l'interface sur le port `3000` (ou `${WEBUI_PORT}`), monte le volume `open-webui` vers `/app/backend/data` , et injecte la variable d'environnement `OLLAMA_BASE_URL: http://ollama:11434` .
- **Réseau** : Les deux services communiquent à travers un réseau de type `bridge` dédié nommé `llm` .
- **Dépendance ordonnée** : La directive `depends_on.condition: service_healthy` garantit qu'Open WebUI ne démarre qu'une fois le serveur Ollama pleinement prêt.

3.2. Téléchargement et inventaire des modèles

Dans le conteneur Ollama, les modèles requis ont été téléchargés en ligne de commande :

```
docker exec -it ollama ollama pull llama3.2:3b
docker exec -it ollama ollama pull mistral:7b
docker exec -it ollama ollama pull gemma3:4b
docker exec -it ollama ollama pull deepseek-r1:8b
```

3.3. Validation et Preuves

```
~/Documents/wk/dic2/cloud-devops/dic2-2026/llm-devops-project -- ssh
[ubuntu@ip-172-31-19-46:~/llm-project]$ docker compose ps
WARN[0000] /home/ubuntu/llm-project/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS        PORTS
ollama              ollama/ollama:latest "/bin/ollama serve"  ollama     2 hours ago   Up 2 hours   (healthy)   0.0.0.0:11434->11434/tcp, [::]:11434->11434/tcp
open-webui          ghcr.io/open-webui/open-webui:main "bash start.sh"      open-webui  2 hours ago   Up 2 hours   (unhealthy)  0.0.0.0:3000->8080/tcp, [::]:3000->8080/tcp

[ubuntu@ip-172-31-19-46:~/llm-project]$
[ubuntu@ip-172-31-19-46:~/llm-project]$ docker exec -it ollama ollama list
NAME                ID                SIZE    MODIFIED
deepseek-r1:8b     6995872bfe4c     5.2 GB  2 hours ago
gemma3:4b          a2af6cc3eb7f     3.3 GB  2 hours ago
mistral:7b         6577883a9a9a     4.4 GB  2 hours ago
llama3.2:3b        a88c4f17acd5     2.0 GB  2 hours ago
[ubuntu@ip-172-31-19-46:~/llm-project]$
```

Figure 1 — Sortie de `docker compose ps` (conteneurs sains) et de `ollama list` (modèles `deepseek-r1`, `gemma3`, `mistral` et `llama3.2`).

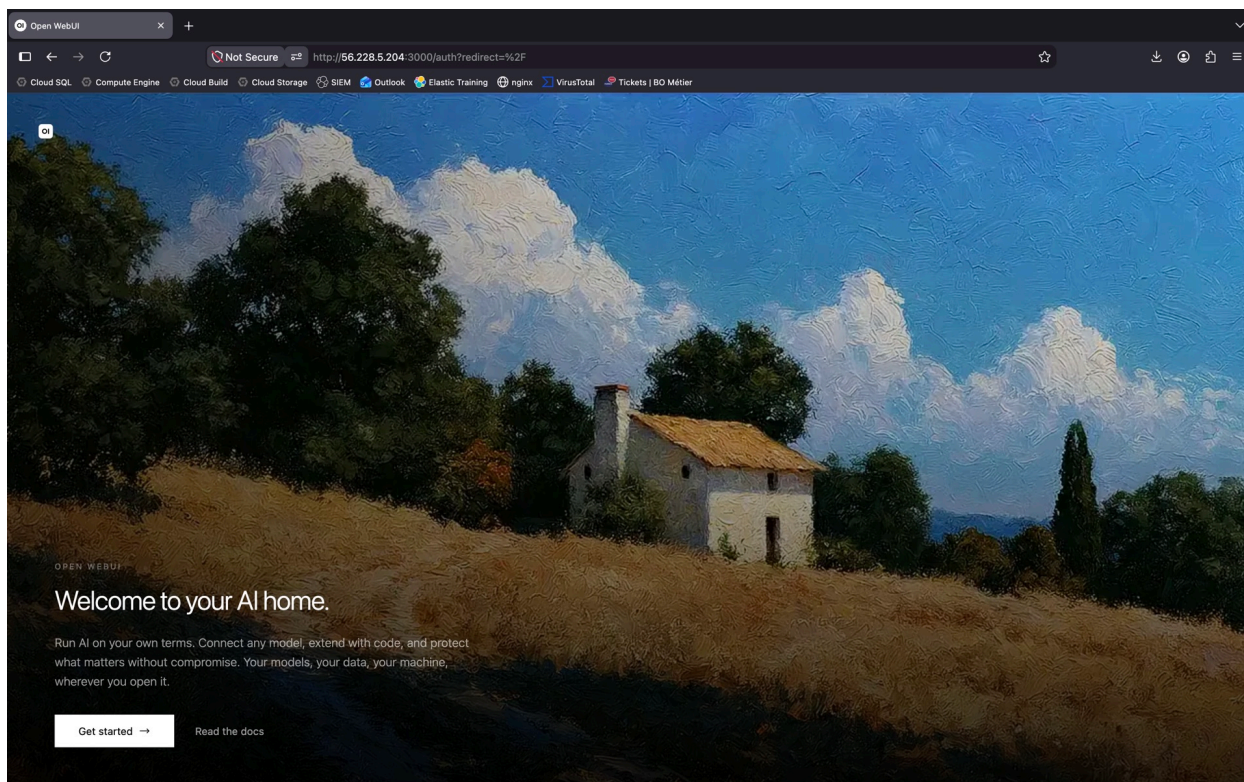


Figure 2 — Page d'accueil et d'authentification d'Open WebUI accessible sur le port 3000.

4. Partie 2 – Migration et Orchestration avec Kubernetes

L'application a été migrée sous forme de manifests déclaratifs dans le répertoire `k8s/`, puis déployée sur un cluster **Minikube**.

4.1. Synthèse des manifests créés

1. `k8s/namespace.yaml` : Namespace dédié `llm`.
2. `k8s/configmap.yaml` : Configuration centralisée `llm-config`.
3. `k8s/ollama-pvc.yaml` : Réclamation de 20 Gi pour les modèles LLM.
4. `k8s/webui-pvc.yaml` : Réclamation de 2 Gi pour les données d'Open WebUI.
5. `k8s/ollama-deployment.yaml` : Pod Ollama avec sondes et limites CPU/RAM.
6. `k8s/ollama-service.yaml` : Service interne `ClusterIP` port 11434.

7. `k8s/webui-deployment.yaml` : Pod Open WebUI avec sondes et injection du ConfigMap.
8. `k8s/webui-service.yaml` : Service externe `NodePort` (port 30080).
9. `k8s/ingress.yaml` : Ingress Controller Nginx pour l'hôte `llm.local`.

4.2. Validation du déploiement

L'exécution des commandes `kubectl get pods -n llm -o wide` et `kubectl get svc -n llm` confirme le statut opérationnel de la pile :

```

~/Documents/wk/dic2/cloud-devops/dic2-2026/llm-devops-project -- zsh
ubuntu@ip-172-31-19-46:~/llm-project$ kubectl get pods -n llm -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE     NOMINATED NODE   READINESS GATES
ollama-7c4f4c484-d6rk5             1/1     Running   0           38m   10.244.0.3      minikube <none>         <none>
open-webui-8584f7a6f9-8f5d7        1/1     Running   0           18m   10.244.0.8      minikube <none>         <none>
ubuntu@ip-172-31-19-46:~/llm-project$ kubectl get svc -n llm
NAME                                TYPE               CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
ollama-service                      ClusterIP        10.201.152.47 <none>        11324/TCP  38m
webui-service                       NodePort         10.100.29.244 <none>        8080:30080/TCP  38m
ubuntu@ip-172-31-19-46:~/llm-project$

```

Figure 3 — Vérification du cluster Kubernetes : Pods `ollama` et `open-webui` au statut `Running` (0 restart) et Services `ClusterIP` / `NodePort` actifs.

5. Analyse Technique Approfondie des Ressources Kubernetes

5.1. Rôle et utilité de chaque ressource

Ressource Kubernetes	Rôle et justification dans l'architecture
Namespace (<code>llm</code>)	Isole logiquement l'ensemble des composants du projet. Cela évite les conflits avec le namespace par défaut, facilite le contrôle d'accès RBAC et permet d'appliquer des quotas de ressources dédiés.
Deployment (<code>ollama</code>)	Gère le cycle de vie du pod Ollama (<code>replicas: 1</code>). Il assure le maintien de l'état désiré, le redémarrage automatique en cas de crash et permet les montées de version sans interruption (<code>RollingUpdate</code>).
Deployment (<code>open-webui</code>)	Assure la résilience du frontend web. Il injecte les variables du ConfigMap et attache le volume persistant contenant les utilisateurs et historiques de chat.

Ressource Kubernetes	Rôle et justification dans l'architecture
Service <code>ollama-service</code> (<code>ClusterIP</code>)	Attribue une IP virtuelle interne fixe et un nom DNS au pod Ollama. Comme les adresses IP des pods sont éphémères, ce service garantit que le frontend joint toujours le backend sans interruption.
Service <code>webui-service</code> (<code>NodePort</code>)	Ouvre le port statique <code>30080</code> sur l'adresse IP du nœud Kubernetes pour rendre l'interface accessible aux postes clients extérieurs.
ConfigMap (<code>llm-config</code>)	Centralise la configuration hors du code conteneurisé (<code>OLLAMA_BASE_URL</code> , <code>OLLAMA_KEEP_ALIVE</code> , etc.). Une modification de paramètre se fait sans reconstruire l'image Docker.
PersistentVolumeClaim (<code>ollama-pvc</code> & <code>webui-pvc</code>)	Alloue un stockage persistant et découplé du cycle de vie des pods auprès du fournisseur de stockage du cluster.
Ingress (<code>llm-ingress</code>)	Gère le routage applicatif HTTP (couche 7) pour l'URL <code>llm.local</code> et permet d'appliquer des directives d'équilibrage et de timeout spécifiques.

5.2. Communication et résolution DNS inter-services

La communication inter-services au sein du cluster fonctionne selon les étapes suivantes :

1. Le pod Open WebUI lit la variable d'environnement `OLLAMA_BASE_URL` issue du ConfigMap :

```
http://ollama-service:11434
```

2. Lors de la transmission d'une requête HTTP, la résolution de nom est déléguée au résolveur DNS interne du cluster (**CoreDNS**).
3. CoreDNS résout le nom d'hôte `ollama-service` vers l'IP virtuelle ClusterIP (`10.101.153.47`).
4. Le composant `kube-proxy` achemine les paquets réseau vers l'IP privée du pod Ollama cible (`10.244.0.3`).

💡 **Avantage clé :** Cette abstraction réseau rend la communication totalement transparente : même si le pod Ollama est détruit et recréé avec une nouvelle IP de pod, le nom DNS `ollama-service` ne change jamais.

5.3. Justification de la persistance des données (PVC)

Dans Kubernetes, le système de fichiers racine d'un conteneur est par défaut **stateless et éphémère** (détruit à la suppression du pod). L'usage de PVC est obligatoire pour deux raisons fondamentales :

1. **Volume des modèles LLM (`ollama-storage` - 20 Gi) :**

Les fichiers de poids des modèles (`deepseek-r1:8b` , `mistral:7b` , `llama3.2:3b`) représentent plus de 15 Go de données. Sans PVC, chaque redémarrage forcerait un re-téléchargement intégral, bloquant le service pendant plusieurs dizaines de minutes.

2. **Pérennité des profils et historiques (`webui-storage` - 2 Gi) :**

Open WebUI enregistre les utilisateurs, les droits d'administration et les conversations dans une base SQLite stockée sur `/app/backend/data` . Le PVC assure qu'aucune donnée utilisateur n'est perdue lors d'une mise à jour du pod web.

5.4. Rôle et utilité des sondes de santé (Health Probes)

Kubernetes dispose de trois sondes complémentaires pour superviser les conteneurs :

1. **Startup Probe (`/api/version` et `/health`)** *Rôle :* Détecte quand l'application a terminé sa phase d'initialisation.

Importance pour les LLM : Le chargement des moteurs d'IA en mémoire vive peut prendre plusieurs dizaines de secondes. La startup probe neutralise les sondes liveness et readiness tant qu'elle n'a pas réussi. Nous avons alloué un `failureThreshold: 30` avec un `periodSeconds: 10` , accordant jusqu'à 5 minutes de tolérance au démarrage.

2. **Readiness Probe** *Rôle :* Indique si le conteneur est apte à recevoir des requêtes utilisateurs.

Comportement : Si Ollama est temporairement saturé par une inférence lourde, la sonde échoue temporairement. Kubernetes retire alors le pod du

Service pour ne pas renvoyer d'erreur 502 aux utilisateurs, puis le réintègre dès qu'il redevient disponible.

3. **Liveness Probe** *Rôle* : Détecte les blocages complets du processus (*deadlocks*).

Comportement : Si le conteneur ne répond plus aux requêtes de santé pendant plusieurs cycles consécutifs, Kubernetes redémarre automatiquement le conteneur pour rétablir le service.

5.5. Étude comparative : Docker Compose vs Kubernetes

Critère d'évaluation	Docker Compose	Kubernetes (K8s)
Environnement cible	Développement local et tests mono-hôte.	Production, environnements distribués et haute disponibilité.
Étendue matérielle	Limité à une seule machine hôte.	Agrégation d'un cluster hétérogène (Control Plane + Workers).
Auto-guérison (<i>Self-Healing</i>)	Redémarrage simple du conteneur local en cas de plantage logiciel.	Détection multi-niveaux (sondes) et reprogrammation sur un autre nœud si le serveur physique crash.
Scalabilité	Manuelle (<code>docker compose up --scale</code>), contrainte par les ressources de l'hôte.	Horizontale et automatique (HPA) basée sur l'usage CPU/RAM ou métriques custom.
Mises à jour applicatives	Coupure de service lors de la recreation du conteneur.	Stratégies avancées de déploiement progressif (<i>Rolling Updates</i>) sans interruption de service.
Stockage persistant	Volumes locaux sur le disque de la machine hôte.	Abstraction dynamique (<code>StorageClass</code> , <code>PV</code> , <code>PVC</code>) compatible cloud et stockage réseau (EBS, NFS, Ceph).
Réseau et Routage	Réseau bridge Docker basique et ports hôtes directs.	Modèle réseau CNI unifié, Services virtuels (ClusterIP/NodePort) et Ingress couche 7.
Complexité d'exploitation	Faible et prise en main immédiate.	Élevée, nécessitant une expertise en orchestration et observabilité.

6. Démonstration et Évaluation des Modèles LLM

Pour répondre à l'exigence de test de trois modèles distincts, chaque modèle a été sollicité sur l'interface Open WebUI :

6.1. Modèle 1 : DeepSeek-R1 (8B) — Raisonnement logique (docker-compose)

Le modèle `deepseek-r1:8b` intègre des mécanismes avancés de réflexion (*Chain of Thought*). Il a été testé avec une question d'organisation créative.

Prompt : *"What are 5 creative things I could do with my kids' art? I don't want to throw them away, but it's also so much clutter."*

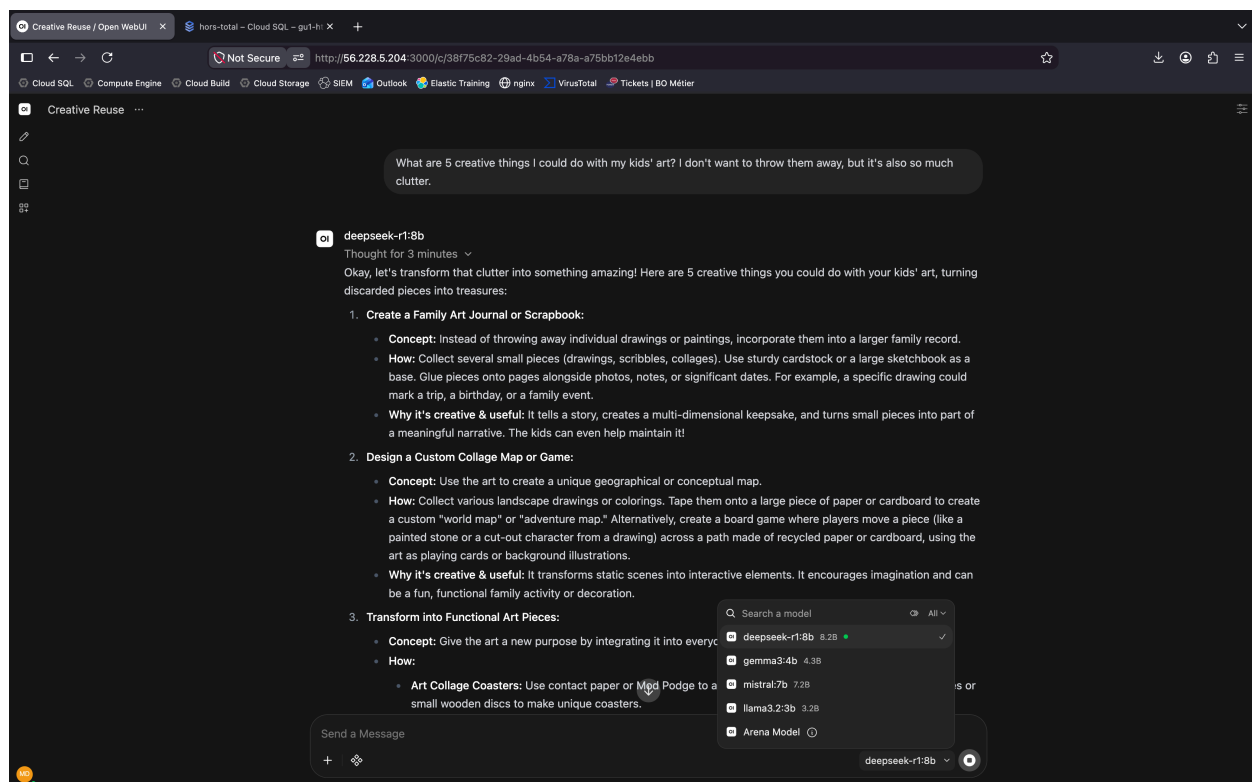


Figure 4 — Inférence de DeepSeek-R1 montrant le bloc de réflexion ("Thought for 3 minutes") et la génération structurée de 5 propositions pratiques.

6.2. Modèle 2 : Mistral (7B) — Tâches généralistes et assistance (k8s)

Le modèle `mistral:7b` se caractérise par son équilibre entre vitesse et compréhension du langage naturel. Il a été testé dans un registre informel.

Prompt : *"Hello my G, what's up gayn ?"*

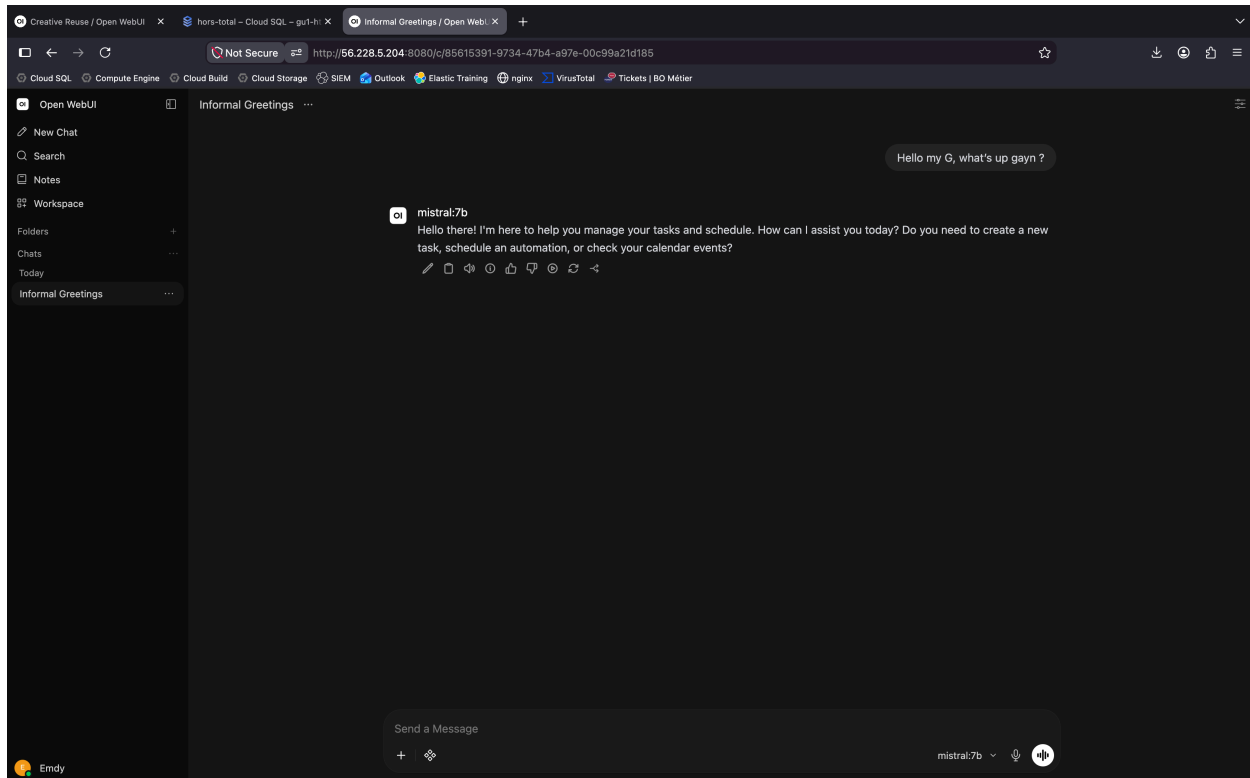


Figure 5 — Réponse contextuelle du modèle Mistral 7B proposant une assistance pour la gestion de tâches et de calendrier.

6.3. Modèle 3 : Llama 3.2 (3B) — Génération de code et réactivité (k8s)

Optimisé pour les faibles latences, le modèle `llama3.2:3b` a été testé sur une tâche de génération de script JavaScript pour la gestion de notes.

Prompt : *"Hello bro llama" puis "How do I save and organize my notes?"*

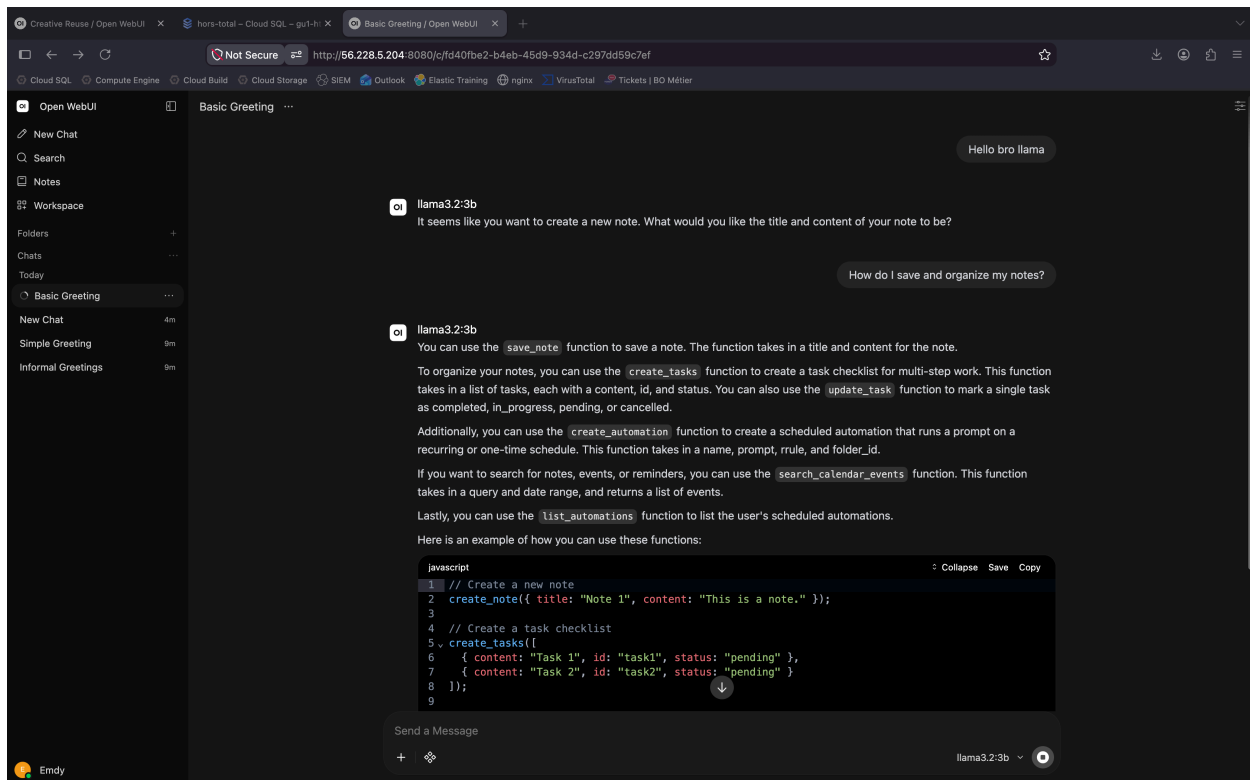


Figure 6 — Génération de code JavaScript structuré et commenté par LLaMA 3.2 pour l'organisation de notes et tâches.

7. Bonnes Pratiques de Production et Fonctionnalités Avancées

7.1. Gestion des ressources (Requests & Limits)

Chaque conteneur dispose de directives `resources` précises :

- **Ollama** : `requests: cpu: 1000m, memory: 2Gi` / `limits: cpu: 4000m, memory: 12Gi`.
- **Open WebUI** : `requests: cpu: 200m, memory: 512Mi` / `limits: cpu: 1000m, memory: 2Gi`.

Justification

- Les **requests** garantissent que le pod ne sera planifié que sur un nœud disposant des capacités nécessaires.
- Les **limits** évitent qu'une inférence volumineuse ne sature toute la mémoire vive du nœud hôte, ce qui déclencherait le tueur de processus de l'OS

| (`OOMKiller`).

7.2. Configuration Ingress Nginx pour le streaming LLM

L'inférence d'un LLM repose sur le protocole **Server-Sent Events (SSE)** qui maintient la connexion HTTP ouverte pendant toute la durée de génération des tokens. Les annotations suivantes ont été incluses dans `k8s/ingress.yaml` :

```
annotations:
  nginx.ingress.kubernetes.io/proxy-read-timeout: "3600"
  nginx.ingress.kubernetes.io/proxy-send-timeout: "3600"
  nginx.ingress.kubernetes.io/proxy-body-size: "0"
```

Elles empêchent Nginx d'interrompre prématurément le flux de réponse lors des requêtes longues.

8. Conclusion

Ce projet a permis de valider avec succès le cycle complet de conteneurisation et d'orchestration d'une charge de travail d'IA moderne :

- **Phase 1** : Prototypage rapide et validation fonctionnelle multi-conteneurs via Docker Compose.
- **Phase 2** : Migration robuste vers Kubernetes avec gestion déclarative, découverte DNS, tolérance aux pannes par sondes de vie, persistance des données et contrôle des ressources.
- **Tests** : Validation opérationnelle de trois modèles de langage complémentaires (`deepseek-r1:8b`, `mistral:7b`, `llama3.2:3b`).
- D'autres modèles ont été testés coté déploiement `docker-compose` e comme pour `kubernetes` aussi (voir dossier screenshots).

L'ensemble des fichiers YAML et des captures d'écran est versionné et consultable sur le dépôt GitHub du projet.